Lightweight ResUMamba-based On-Device Real-Time Arrhythmia Classification for Wearable ECG Monitoring

Ryuha Kim

Department of Computer Science
Graduate School

Yonsei University

Lightweight ResUMamba-based On-Device Real-Time Arrhythmia Classification for Wearable ECG Monitoring

Advisor: Youngcheol Park

A Master's Thesis Submitted
to the Department of Computer Science
and the Committee of the Graduate School
of Yonsei University in Partial Fulfillment of the
Requirements for the Degree of
Master in Computer Science

Ryuha Kim

February 2027

Lightweight ResUMamba-based On-Device Real-Time Arrhythmia Classification for Wearable ECG Monitoring

This certifies that the dissertation
of Ryuha Kim is approved.

Committee Chair: Youngcheol Park

Committee Member: Hanul Yoon

Committee Member: Jaekwon Kim

Department of Computer Science
Graduate School
Yonsei University
February 2027

감사의 글

Contents

List of Figures ..... **iii**

List of Tables ..... **iv**

Abstract ..... **v**

Chapter 1. Introduction **1**

Chapter 2. Background and Related Works **6**

2.1 Inter-Patient Arrhythmia Classification from Wearable ECG 6

2.2 Patient-Conditioned ResUMamba 7

2.3 On-Device ECG Classification and Mobile Deployment 10

Chapter 3. Model Optimization and Deployment for On-Device Inference ... 13

3.1 Optimization Targets and Constraints 13

3.2 ResU Path: Depthwise Separable Convolution 15

3.3 Mamba Path: Rewriting the Selective Scan for Mobile Deployment 19

Chapter 4. On-Device Real-Time Implementation 22

4.1 ECG Data Acquisition 22

4.2 On-Device Processing 24

Chapter 5. Experiments 28

5.1 Experimental Setup 28

5.1.1 Real-World Wearable ECG Dataset 28

5.1.2 Target Device and Measurement 29

5.1.3 Evaluation Metrics 30

5.2 Experimental Results 32

5.2.1 Arrhythmia Classification Performance 32

5.2.2 Effect of Software Optimization on Mobile CPU Time 33

5.2.3 Real-World Live Wearable Patch Evaluation 34

Chapter 6. Conclusion 37

6.1 Limitations and Future Work 39

References **40**

Appendix A. Evaluation of Pruning for Model Lightweighting 48

List of Figures

Figure 2.1 Dual-path backbone of patient-conditioned ResUMamba [19]. 8

Figure 3.1 Overall system architecture from wearable ECG acquisition to optimized on-device inference. 14

Figure 3.2 Depthwise separable convolution [42]. 15

Figure 3.3 Single Res-U block ($L = 2$). 17

Figure 3.4 Selective SSM (S6) of the Mamba path. 21

Figure 4.1 HiCardi® Smart Patch and system configuration. 23

Figure 4.2 BLE frame structure and ECG sample encoding. 24

Figure 4.3 End-to-end on-device beat processing. 25

Figure 5.1 Resource usage during live ECG streaming. 36

Figure A.1 Weighted F_1 versus prune ratio. 49

Figure A.2 Mobile CPU latency versus prune ratio. 50

List of Tables

Table 3.1 Parameters and module-wise MACs. 19

Table 5.1 Cohort composition of the HiCardi records. 28

Table 5.2 Classification performance on the HiCardi test set. 32

Table 5.3 Model execution time under Portable and XNNPACK configurations. . . 33

Table 5.4 Resource usage during continuous ECG streaming. 35

ABSTRACT

Lightweight ResUMamba-based On-Device Real-Time Arrhythmia Classification for Wearable ECG Monitoring

Ryuha Kim
Department of Computer Science
The Graduate School, Yonsei University, Korea

Wearable ECG patches enable long-term monitoring of intermittent arrhythmias during everyday life, but long recordings are difficult to review manually, and cloud-based analysis introduces additional communication and data-processing steps. This thesis deploys the existing patient-conditioned ResUMamba beat classifier on a smartphone and evaluates a lightweight model variant together with a CPU-oriented deployment approach. The ResU path is modified by replacing standard convolutions with depthwise separable convolutions and replacing the decoder's transposed convolutions with parameter-free linear interpolation, yielding ResDSUMamba, while the Mamba selective scan is rewritten using standard PyTorch ATen operators and represented as a sequential

graph unrolled over the fixed input length of $T = 501$. No new classifier architecture is introduced.

ResDSUMamba has 1.3225 M parameters and 331.38 M MACs, compared with 2.4883 M and 721.50 M for ResUMamba, corresponding to reductions of 46.9% and 54.1%, respectively. On a record-wise split of the HiCardi dataset, the Macro-F_1 and Micro-F_1 of ResDSUMamba are 44.15% and 83.29%, compared with 42.28% and 79.27% for ResUMamba, while Macro-Se decreases from 78.00% to 75.72%. On a Galaxy S21+ (Exynos 2100) using a single Cortex-X1 thread in FP32, XNNPACK delegation reduces the mean T_{model} of ResUMamba from 1073.57 ms to 48.25 ms, and DSConv reduces it further to 36.70 ms for ResDSUMamba, with a P95 of 38.90 ms. In a live patch-connected session lasting 309.21 s, 413 beats were processed at a reported mean heart rate of 80.2 bpm without backlog. The mean T_{beat} was 59.4 ms with a P95 of 70.3 ms, while inference-thread CPU utilization was 7.9% and peak application-level native heap usage was 99.2 MB. The live session had no reference labels, so on-device classification accuracy was not evaluated.

Key words : Electrocardiogram (ECG), Arrhythmia Classification, State-Space Models (SSMs), On-Device Inference, Mobile CPU Optimization, Wearable Healthcare

Chapter 1

Introduction

Cardiovascular disease (CVD) is one of the leading causes of death worldwide, making early diagnosis and continuous monitoring important. Among cardiovascular disorders, arrhythmia, which arises from irregular electrical activity of the heart, can lead to serious complications such as hemodynamic instability, heart failure, and cardiac arrest. Atrial fibrillation (AFIB), in particular, is a common form of arrhythmia and has been reported to increase the risk of stroke by approximately fivefold [1]. Paroxysmal arrhythmias are intermittent and often short-lived, so a standard 12-lead electrocardiogram (ECG) recorded for approximately $10\,\mathrm{s}$ may not capture every episode. For this reason, long-term ECG monitoring with wearable patches has been used to detect intermittent arrhythmias [2].

Irregular pulse detection using smartwatches, followed by confirmatory ECG recording with an ECG patch, is another approach used for out-of-hospital arrhythmia detection [3]. Continuous ECG monitoring with wearable patches has also been reported [2]. These devices can record cardiac activity with relatively little disruption to daily life, making them useful for long-term monitoring. At the same time, reviewing the large amount of ECG data accumulated over several days or weeks is difficult to perform

manually. Automated ECG analysis is therefore needed to support long-term monitoring.

Early studies on automated ECG classification relied mainly on conventional signal-processing methods. Typical approaches extracted delineation features such as beat intervals, QRS duration, and waveform amplitude, or manually designed morphological and temporal features and then used them as inputs to machine-learning classifiers [4–7]. Deep neural networks (DNNs) later became the dominant approach because they can learn feature extraction directly from the data in an end-to-end manner. Representative models include convolutional neural networks (CNNs), which are effective at extracting local morphological patterns [8, 9], recurrent neural networks (RNNs) [10, 11] and state space models (SSMs) [12,13], which model temporal dependencies, and transformers [14, 15], which capture relationships across time steps.

Despite their high classification performance on known data, DNN models can degrade substantially when they are applied to previously unseen patients [16, 17]. ECG morphology depends not only on cardiac anatomy and physiological state but also on factors such as age, sex, underlying disease, electrode placement, and recording-device characteristics. Consequently, a model may perform well in an intra-patient setting, where the same patient's beats appear in both training and evaluation data, but perform worse in an inter-patient setting, where the evaluation data contain patients not seen during training.

To address this variability, personalization methods that incorporate patient-specific baseline information into the model have been proposed [18–20]. Rather than relying only on absolute waveform morphology, personalization allows the model to focus on

deviations from the patient's own baseline, which can reduce the effect of inter-patient morphological variation. The main categories of personalization methods and the position of the target model are reviewed in Section 2.1.

Most previous studies have focused on classification accuracy and personalization performance. In comparison, relatively few studies have examined deep-learning models together with on-device streaming on mobile hardware and measurements obtained from an actual device. A patient-conditioned architecture also introduces additional computation for extracting patient characteristics and combining them with classification features. The original Mamba implementation provides a CUDA-accelerated selective-scan path as well as a pure PyTorch reference implementation [21]. For mobile CPU deployment, this work uses a sequential scan implemented with standard ATen operators.

For wearable and mobile systems, it is not enough to classify isolated ECG segments; the system must continue processing beats as the ECG stream arrives. In cloud-based analysis, raw ECG must be transmitted to a server, introducing an external communication path and computing dependency [22]. In contrast, on-device analysis performs ECG processing and inference directly on the target device. In this work, real-time processing is defined as continuously constructing beat-level inputs from the incoming ECG stream and running classification inference for each beat. Both model execution time and the processing time observed during actual streaming are measured.

The target model is patient-conditioned ResUMamba [19], which was proposed to improve inter-patient generalization. This work adapts the model for beat-level execution on a smartphone through model lightweighting and deployment-oriented changes. The convolutions in the ResU path are replaced with depthwise separable convolutions

to reduce the model size and computational cost. The selective scan in the Mamba path is rewritten into a form that can be executed using standard mobile CPU operators. A smartphone application is also implemented to communicate with the HiCardi® Smart Patch and to measure the complete streaming path from ECG reception to beat-level inference. The experiments compare ResUMamba and ResDSUMamba using a record-wise split of the HiCardi® dataset, and measure model execution time, streaming processing time, and resource usage on a Galaxy S21+.

The contributions of this work are as follows.

- ResDSUMamba is constructed by replacing the standard convolutions in the ResU path of the existing ResUMamba with depthwise separable convolutions (DSConv) and replacing the decoder's transposed convolutions with parameter-free linear interpolation, followed by retraining on the HiCardi dataset.

- The selective scan in the Mamba path is implemented as a sequential scan using standard PyTorch ATen operators and converted into a form that can run with ExecuTorch and the XNNPACK backend. Execution time is measured under different backend configurations.

- A smartphone application is implemented to communicate with the HiCardi patch over BLE. Sample indices are used as the time axis, a circular buffer is used to construct beat segments, and ECG reception and inference are handled on separate threads.

- The classification performance of ResUMamba and ResDSUMamba is compared on the HiCardi dataset, and model execution time, per-beat processing time during

streaming, and resource usage are measured on an actual smartphone.

Chapter 2

Background and Related Works

This chapter reviews the studies that form the basis of this work. First, beat-level arrhythmia classification from wearable ECG and the problem of inter-patient variability are introduced, followed by the architecture of the patient-conditioned ResUMamba model. Next, previous work on embedded and mobile arrhythmia classifiers is reviewed. Finally, mobile DNN deployment frameworks and selective-scan implementations are discussed, with particular attention to the cost of parallel selective scan on a single-threaded CPU.

2.1 Inter-Patient Arrhythmia Classification from Wearable ECG

Approaches to inter-patient variability can be broadly divided into two groups. The first is a generalization approach, which learns features that are shared across patients [4, 8, 17]. The second is a personalization approach, which explicitly incorporates patient-specific information.

Among personalization methods, training a separate classifier for each patient [20, 23, 24] or fine-tuning a pretrained model with patient-specific data requires additional

annotation and training for each new patient. Such procedures can be difficult to apply in practical deployment. An alternative is to extract an embedding that represents the patient and combine it with intermediate classification features. For example, i-vector-based methods originally developed for speaker recognition have been adapted to ECG [18]. Because a single model can then be shared across patients, this approach is more suitable for deployment.

Label-free patient conditioning [19] takes this idea one step further by extracting a patient embedding from a baseline ECG without using arrhythmia labels. The same model can then be used for a new patient without additional labeling or fine-tuning after deployment. This makes the approach particularly attractive for wearable applications. The ResUMamba model studied in this work follows this approach.

2.2 Patient-Conditioned ResUMamba

The target model of the real-time on-device implementation is the patient-conditioned ResUMamba model proposed for patient-conditioned beat classification [19]. As shown in Fig. 2.1(a), the model uses a dual-path backbone in which an R-peak-aligned beat segment $\tilde{\mathbf{u}}$ is processed in parallel by the ResU and Mamba paths. The ResU path extracts morphological features from structures such as the P wave, QRS complex, and T wave using multiresolution convolutions, while the Mamba path models temporal dependencies within the beat using a selective state space model. The outputs of the two paths are concatenated and subsequently used for patient conditioning with a patient embedding vector (PEV) $\mathbf{p}$ obtained from the patient's baseline ECG. The conditioned representation is then combined with clinical features $\mathbf{c}$, such as RR intervals,

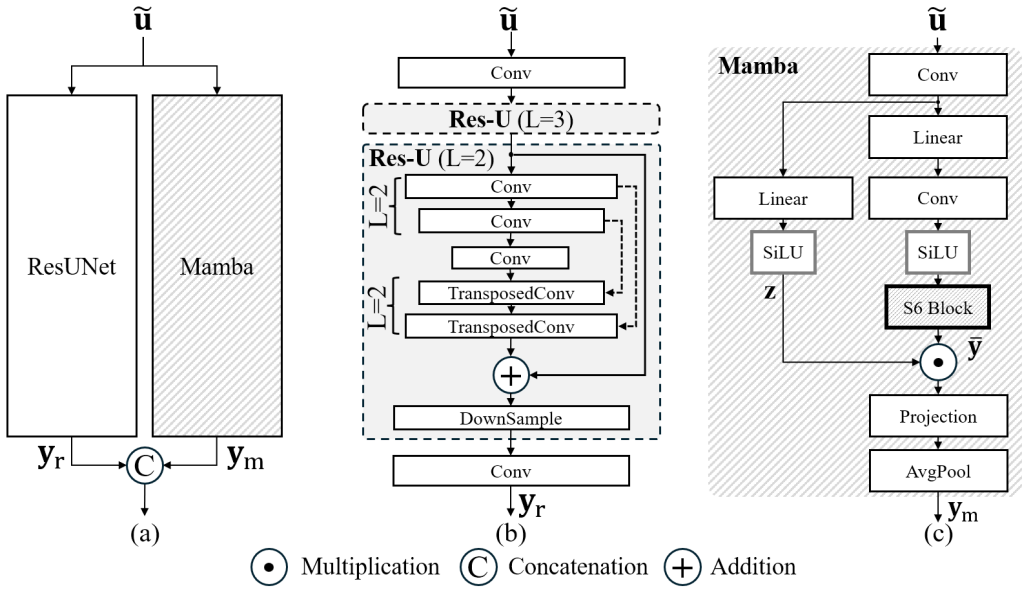


Figure 2.1: Dual-path backbone of patient-conditioned ResUMamba [19]. (a) Parallel ResU and Mamba paths. (b) ResU path with two cascaded Res-U blocks. (c) Gated Mamba path with an S6 block.

before being passed to the classifier.

The ResU path is a residual U-Net [25, 26]-based architecture adapted for one-dimensional ECG. As shown in Fig. 2.1(b), it consists of an input convolution, Res-U blocks with depths $L = 3$ and $L = 2$, and an output convolution. Within each block, the temporal resolution is reduced through L stride-2 convolutions, restored using transposed convolutions and skip connections, and then combined with the block input through a residual connection. The resulting representation is downsampled before entering the next stage. Because the path repeatedly uses standard one-dimensional convolutions, these operations account for a large part of the computational cost on a mobile device.

The Mamba path is based on a state space model (SSM). An SSM discretizes a

continuous-time linear system and updates its state with a computational cost that grows linearly with sequence length. The Structured State Spaces for Sequence (S4) Family [27,28] established the use of structured state-transition matrices, including diagonal parameterizations. Mamba [21] introduced the selective state space model (S6), in which the discretization step and the input and output projections depend on the current input. This allows the model to selectively retain information in its state. As shown in Fig. 2.1(c), ResUMamba uses an input convolution followed by a gating branch (Linear–SiLU) and a main branch (Linear–Conv–SiLU–S6). The outputs of the two branches are multiplied element-wise, then projected and average-pooled to obtain $\mathbf{y}_m$. Let the input feature at time t be $\mathbf{x}_t \in \mathbb{R}^D$ and the state be $\mathbf{H}_t \in \mathbb{R}^{D\times N}$. The S6 update is given by

$$\begin{aligned} \mathbf{H}_t &= \overline{\mathbf{A}}_t \odot \mathbf{H}_{t-1} + \mathbf{b}_t, \qquad \mathbf{b}_t = (\boldsymbol{\delta}_t \odot \mathbf{x}_t)\mathbf{B}_t^\top, \\ \mathbf{y}_t &= \mathbf{H}_t\mathbf{C}_t + \mathbf{d} \odot \mathbf{x}_t. \end{aligned} \tag{2.1}$$

Here, $\boldsymbol{\delta}_t \in \mathbb{R}^D$ denotes the discretization step, and $\mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^N$ are the vectors that write the input into the state and read the state into the output, respectively. All three are generated from $\mathbf{x}_t$. $\mathbf{A} \in \mathbb{R}^{D\times N}$ contains the channel-wise diagonal state-transition parameters. Accordingly, the discretized transition is defined as $\overline{\mathbf{A}}_t = \exp(\boldsymbol{\delta}_t \mathbf{1}_N^\top \odot \mathbf{A})$, where $\mathbf{d} \in \mathbb{R}^D$ is the skip coefficient. The operator $\odot$ denotes element-wise multiplication, and the computational cost for a sequence of length T is $\mathcal{O}(TDN)$. Because $\mathbf{H}_t$ depends on $\mathbf{H}_{t-1}$, directly evaluating Eq. (2.1) requires T sequential update steps.

The PEV $\mathbf{p}$ provides the patient-conditioning information used by the classifier. It is obtained from a short, label-free calibration ECG segment using an encoder trained with contrastive predictive coding (CPC)-based self-supervised learning [29]. The encoder maps the calibration segment to a fixed-dimensional representation that summa-

rizes patient-specific ECG characteristics. Because the calibration procedure does not require arrhythmia labels, the same procedure can be applied to a new patient without additional labeling or model fine-tuning.

2.3 On-Device ECG Classification and Mobile Deployment

Real-time arrhythmia classification for wearable and mobile devices has been investigated using CNN-, RNN-, and TCN-based models, including implementations on wearable platforms and microcontrollers [30–33]. More recently, state-space model (SSM)-based approaches have also been explored for ECG analysis [12, 13], and inter-patient arrhythmia classifiers have been investigated for edge-oriented deployment [34]. A real-time ECG service based on HiCardi uses a smartphone for data transfer and user interaction, while signal analysis is performed on a server [22]. In contrast, the present work targets on-device execution of the patient-conditioned ResUMamba model, including the complete streaming path from BLE reception and buffering to R-peak alignment and beat-level inference.

On-device execution requires a model representation and runtime that are supported by the target mobile platform. TensorFlow Lite [35], ExecuTorch [36], and ONNX Runtime [37] provide mobile or edge execution frameworks for trained neural networks. In the PyTorch ecosystem, ExecuTorch uses `torch.export` to obtain an exportable computational graph and supports delegation of supported subgraphs to optimized backend implementations. XNNPACK [38] provides optimized kernels for mobile CPU execution, including common operations such as convolution and linear algebra, and can be used as a backend in both TensorFlow Lite and ExecuTorch. In ExecuTorch, oper-

ations supported by XNNPACK are executed through the delegated backend, whereas unsupported operations remain on the portable-kernel path. This distinction is important because the practical latency of a model depends not only on its parameter count or theoretical computational cost, but also on how its operators are supported by the target runtime and hardware.

This consideration is particularly important for SSM-based models. The official Mamba implementation provides a CUDA-accelerated selective-scan implementation together with a reference PyTorch path [21]. The CUDA extension is not available on the Android CPU target considered in this work. Therefore, the selective SSM recurrence in Eq. (2.1) is expressed using standard ATen operators so that it can be represented in the exported graph and executed on the mobile CPU. Although parallel prefix-scan algorithms can reduce sequential depth on parallel hardware [39–41], they were not adopted here because the target deployment uses a single CPU execution thread and the study focuses on a standard operator graph rather than a dedicated scan implementation.

Model-level lightweighting techniques such as depthwise separable convolution, quantization, pruning, and knowledge distillation can reduce computational cost [42–44]. However, a reduction in theoretical computation does not necessarily translate into lower latency when the resulting operators are not efficiently supported by the target runtime. Related SSM deployment studies have therefore explored specialized operator implementations or dedicated hardware, such as MCU-oriented MambaLite-Micro [45] and NPU-oriented XAMBA [46]. Unlike these approaches, the present work focuses on smartphone CPU execution using standard ExecuTorch operators and XNNPACK delegation, with model lightweighting and runtime support considered together for practical

on-device ECG inference.

Chapter 3

Model Optimization and Deployment for On-Device Inference

This chapter describes the model-level optimization and deployment methods used to execute ResUMamba in real time on a smartphone. The overall system architecture is shown in Fig. 3.1. Section 3.1 defines the optimization objectives and constraints, and Section 3.2 applies depthwise separable convolution to the ResU path. Section 3.3 rewrites the selective scan in the Mamba path as a sequential scan executable on the mobile runtime. The patient-conditioning mechanism, clinical-feature fusion, model input and output, and training procedure remain unchanged.

3.1 Optimization Targets and Constraints

The goal of this work is to run ResUMamba on a smartphone connected to a wearable patch and process the ECG on a beat-by-beat basis. To support this deployment, the number of parameters and MACs is reduced, and CPU execution time is measured on the target device.

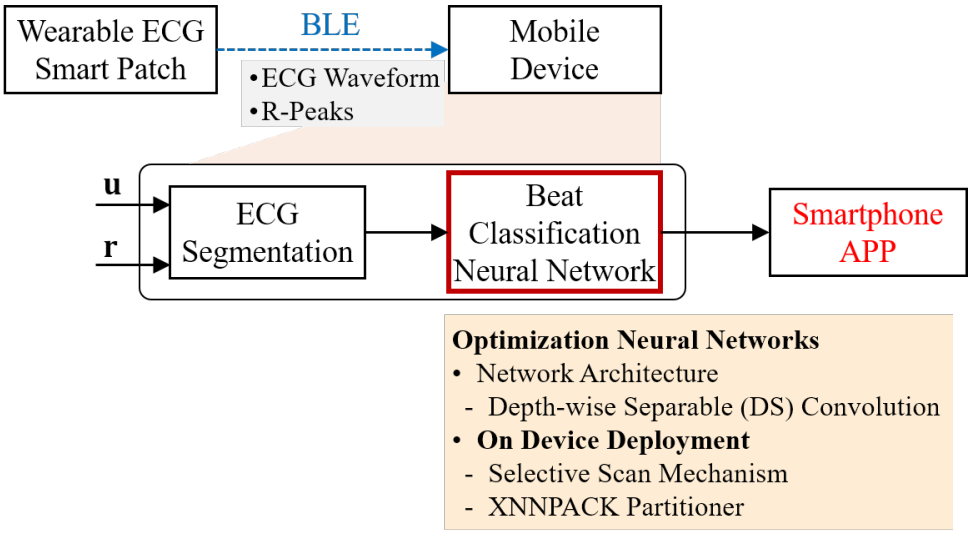


Figure 3.1: Overall system architecture from wearable ECG acquisition to optimized on-device inference.

The CPU execution time of a single forward pass with a prepared beat input is denoted by T_{model} and is used as the main model-level timing metric. In a streaming setting, however, model execution is only part of the total processing path. Each beat must be processed before subsequent inputs accumulate. We therefore report the mean, $P5$, $P50$, and $P95$ of T_{model} and additionally measure T_{beat} and T_{queue} during live streaming. The R-peak detector uses a minimum interval of $240\,\text{ms}$ between consecutive R-peaks.

The ResU path repeatedly applies standard convolutions and therefore accounts for a substantial portion of the model's computational cost. These convolutions are replaced with depthwise separable convolutions to reduce both computation and parameter count. The resulting ResDSUMamba model is retrained using the same HiCardi classification setting. The Mamba path then uses the weights of the retrained ResDSUMamba, and its computational representation is converted to an ATen-based sequential scan during ex-

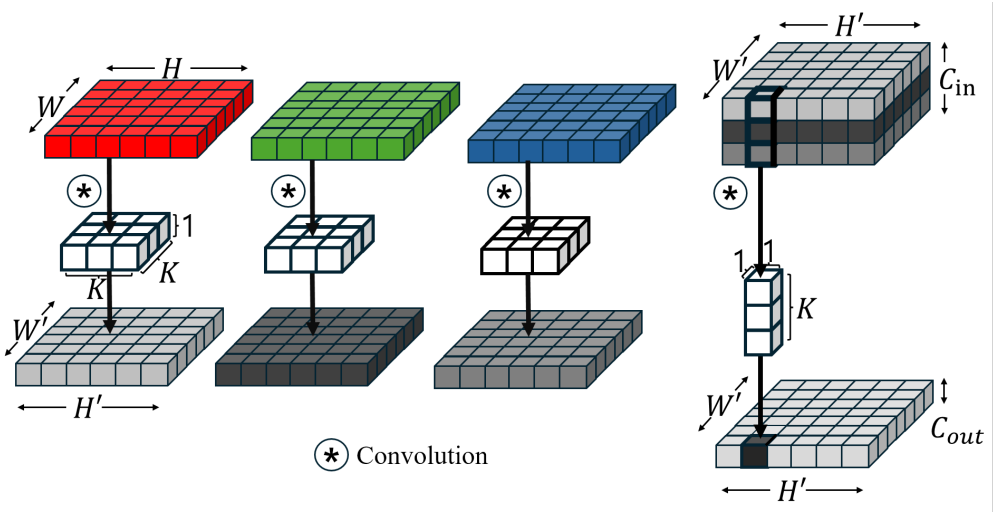


Figure 3.2: Depthwise separable convolution: independent channel-wise filtering followed by 1×1 pointwise convolution for channel mixing.

port. Pruning was also evaluated as an alternative lightweighting method and was found unsuitable for this deployment target; the corresponding experiments and the rationale for instead adopting DSConv are reported in Appendix A.

3.2 ResU Path: Depthwise Separable Convolution

Depthwise separable convolution (DSConv) [42] separates the temporal filtering and channel mixing performed jointly by a standard convolution into two stages. MobileNets adopted this structure as a core building block for mobile vision models and helped popularize lightweight convolutional architectures. As shown in Fig. 3.2, DSConv consists of a depthwise convolution that applies an independent filter to each input channel and a pointwise convolution with kernel size 1 that mixes information across channels. Let the input and output channel vectors at time t be $\mathbf{x}_t \in \mathbb{R}^{C_{\text{in}}}$ and $\mathbf{y}_t \in \mathbb{R}^{C_{\text{out}}}$, respectively. A

standard one-dimensional convolution with kernel size K is defined as

$$\mathbf{y}_t = \sum_{j=0}^{K-1} \mathbf{W}_j \mathbf{x}_{t-j}, \qquad \mathbf{W}_j \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}, \tag{3.1}$$

where stride and padding are omitted for clarity. At each time step, a matrix-vector product of size $C_{\text{out}} \times C_{\text{in}}$ is performed K times. Thus, excluding bias terms, the number of parameters P_{conv} and multiply-accumulate (MAC) operations M_{conv} for an output length $\tilde{T}$ are

$$P_{\text{conv}} = KC_{\text{in}}C_{\text{out}}, \qquad M_{\text{conv}} = \tilde{T}KC_{\text{in}}C_{\text{out}}. \tag{3.2}$$

DSConv first applies an independent K-tap temporal filter to each input channel:

$$\mathbf{z}_t = \sum_{j=0}^{K-1} \mathbf{g}_j \odot \mathbf{x}_{t-j}, \qquad \mathbf{g}_j \in \mathbb{R}^{C_{\text{in}}}, \tag{3.3}$$

where $\odot$ denotes channel-wise multiplication and no channel mixing is performed. A pointwise convolution then combines channel information at the same time step using a matrix $\mathbf{Q}$:

$$\mathbf{y}_t = \mathbf{Q}\mathbf{z}_t, \qquad \mathbf{Q} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}. \tag{3.4}$$

Therefore, the number of parameters and MAC operations for DSConv are

$$P_{\text{DS}} = KC_{\text{in}} + C_{\text{in}}C_{\text{out}}, \qquad M_{\text{DS}} = \tilde{T}\left(KC_{\text{in}} + C_{\text{in}}C_{\text{out}}\right). \tag{3.5}$$

The relative computational cost of DSConv compared with a standard convolution is $M_{\text{DS}}/M_{\text{conv}} = 1/C_{\text{out}} + 1/K$. For example, with $K = 7$ and $C_{\text{out}} = 128$, a standard convolution needs $7 \cdot 128 = 896$ MACs per input channel and output step, whereas DSConv needs $7 + 128 = 135$, which is 15.1% of the original, a reduction of $1 - (1/128 + 1/7) \approx 84.9\%$. In this work, the convolutions of the encoder blocks, bottleneck block, and decoder blocks of the ResU path in Fig. 3.3 are replaced with DSConv, and

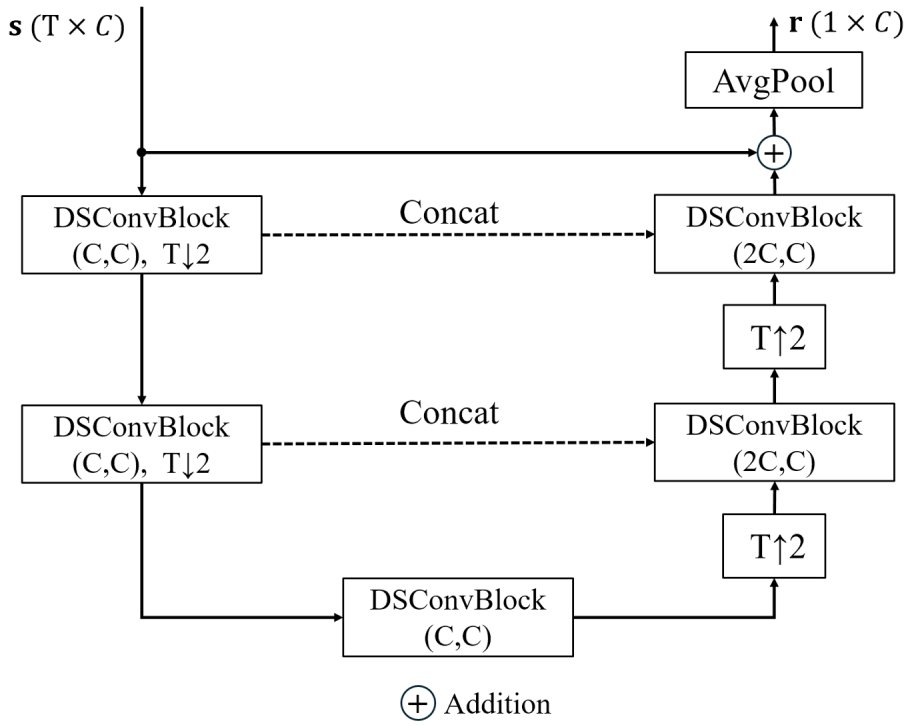


Figure 3.3: A single Res-U block with encoder–decoder layer depth $L = 2$, skip connections, residual addition, and average pooling to produce $\mathbf{y}_r$. The ResU path cascades two such blocks with depths $L = 3$ and $L = 2$ (Section 2.2). T↑2: parameter-free linear interpolation.

these convolutions use $K = 7$ and $C_{\text{out}} = 128$. In the implementation used in this work, the decoder restores the temporal length with parameter-free linear interpolation (T↑2 in Fig. 3.3) instead of the transposed convolutions of the original model, and the decoder convolutions that follow the skip concatenation are replaced with DSConv.

Table 3.1 summarizes the parameter counts and module-wise MACs of the two models for an input length of 501. MACs include only multiply-accumulate operations in convolutions, linear layers, matrix multiplications, including batched matrix multiplications, and attention. Element-wise operations, including normalization, activation func-

tions, and the element-wise state update in the scan, $\overline{\mathbf{A}}_t \odot \mathbf{H}_{t-1} + \mathbf{b}_t$, are excluded. The MAC count of the ResU path decreases from $490.27\,\mathrm{M}$ to $100.15\,\mathrm{M}$, a reduction of 79.6%, which is smaller than the layer-level value of 84.9%. Because the ratio $1/C_{\mathrm{out}} + 1/K$ does not depend on C_{in}, the decoder convolutions that receive the $2C$-channel concatenation reach the same layer-level reduction as the other replaced convolutions. The gap therefore comes from the operations that are not replaced: the downsampling layers between Res-U blocks and the kernel-size-1 residual shortcut convolutions, which cannot be factorized further. Their MACs are the same in both models, so they make up a larger share of the reduced total. The replacement of the decoder's transposed convolutions with parameter-free linear interpolation removes additional MACs beyond the per-layer DSConv substitution, but this interpolation cost is small relative to the convolutional layers, so the unreplaced downsampling and shortcut convolutions remain the dominant contributor to the gap between the measured 79.6% reduction and the 84.9% layer-level value. The Mamba path, patient conditioning, feature fusion, and classifier are unchanged. Consequently, the total MAC count of the model decreases by 54.1% ($721.50\,\mathrm{M} \rightarrow 331.38\,\mathrm{M}$), and the parameter count decreases by 46.9% ($2.4883\,\mathrm{M} \rightarrow 1.3225\,\mathrm{M}$).

Depthwise convolution performs relatively many memory accesses compared with its arithmetic workload and therefore has low arithmetic intensity. Consequently, the reduction in theoretical operation count does not necessarily translate directly into an equivalent reduction in actual execution time. In addition, depthwise and pointwise convolutions have different data layouts and tiling characteristics, making fusion into a single kernel difficult in general-purpose CPU libraries. An intermediate tensor must there-

Table 3.1: Parameters and module-wise MACs (input length 501, batch 1). Element-wise operations, including the scan state update, are excluded. The cross-attention row counts only the attention-weight matrix multiplications ($\mathbf{QK}^\top$ and the attention-weighted sum), which have no associated trainable parameters and are therefore omitted from the parameter rows.

Module	ResUMamba	ResDSUMamba	Δ
MACs (M)			
Input convolution	0.07	0.07	–
ResU path	490.27	100.15	-79.6%
Mamba path	129.28	129.28	–
Patient conditioning	0.26	0.26	–
Fusion, classifier	98.78	98.78	–
Cross-attention	2.84	2.84	–
Total	721.50	331.38	-54.1%
Parameters (M)			
Input convolution	0.0004	0.0004	–
ResU path	1.4787	0.3128	-78.8%
Mamba path	0.2653	0.2653	–
Patient conditioning	0.2632	0.2632	–
Fusion, classifier	0.4807	0.4807	–
Total	2.4883	1.3225	-46.9%

fore be materialized between the two stages. The excluded element-wise scan update also remains unchanged by DSConv. The resulting change in execution time on the target device is evaluated in Section 5.2.2.

3.3 Mamba Path: Rewriting the Selective Scan for Mobile Deployment

Mamba provides an official CUDA-accelerated selective-scan path, but the selective SSM itself can be represented using standard tensor operations [21]. Because CUDA

extensions cannot be used for Android CPU deployment, this work uses a deployment-oriented sequential scan that constructs the recurrence in Eq. (2.1) from standard PyTorch ATen operators. DSConv is applied during training, whereas the selective scan is transformed during export. The export stage uses the trained weights and converts the computational representation into ATen operations. This implementation is referred to below as the deployment-oriented sequential scan.

Fig. 3.4 illustrates the computation of the selective SSM. From the input $\mathbf{x}_t$, three linear projections compute $\mathbf{B}_t$, $\mathbf{C}_t$, and $\boldsymbol{\delta}_t$. $\mathbf{x}_t$ itself is used again in $\mathbf{b}_t$ and in the skip term $\mathbf{d} \odot \mathbf{x}_t$. These quantities are used to construct $\overline{\mathbf{A}}_t$ and $\mathbf{b}_t$. Because these computations depend only on the input at each time step, they can be computed independently for the entire sequence. Accordingly, the sequential scan first computes $\overline{\mathbf{A}}$ and $\mathbf{b}$ for the entire sequence and then performs only the state update sequentially in the order $t = 1, 2, \ldots, T$. As discussed in Section 2.3, a parallel prefix scan is not expected to shorten execution on a single thread, because its stages still run one after another and its total work is the same or larger. Therefore, this sequential execution is used as the deployment implementation in this work. Because the input length is fixed at $T = 501$, `torch.export` unrolls the loop into a static graph with 501 time steps, and the resulting graph consists of standard PyTorch operators.

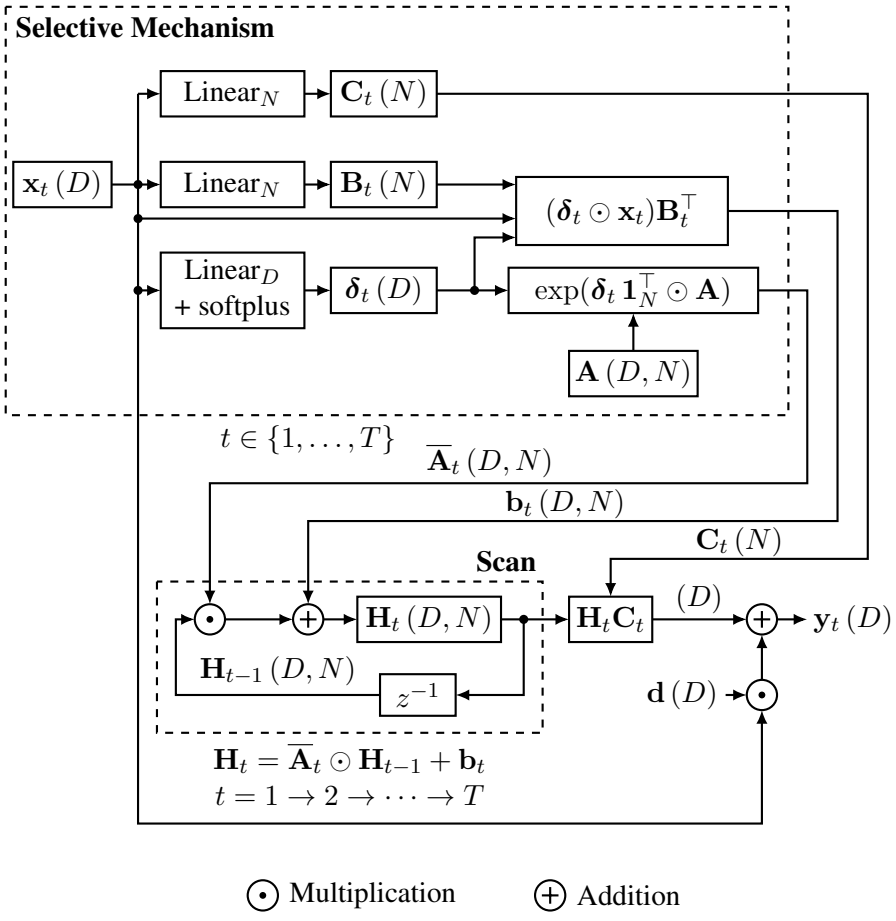


Figure 3.4: Selective SSM (S6) of the Mamba path. All terms except the state update are computed independently at each step; only the state update is sequential. z^{-1}: one-step delay.

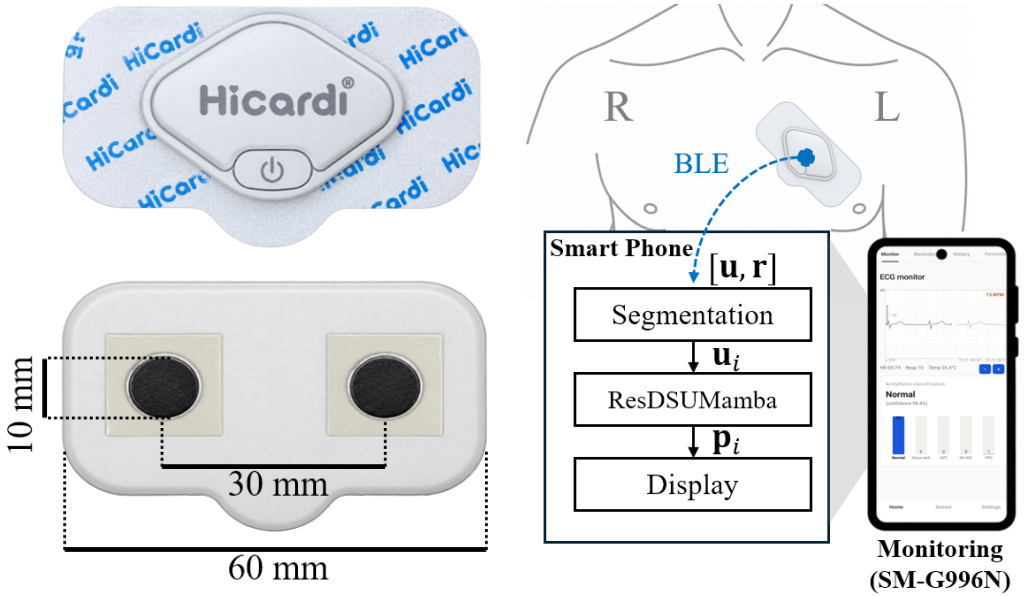
Chapter 4

On-Device Real-Time Implementation

The ResDSUMamba deployed in Chapter 3 was integrated into a smartphone application that continuously receives single-lead ECG from the HiCardi® Smart Patch and processes the signal on a beat-by-beat basis. In this work, real-time processing means continuously constructing beat-level inputs from the incoming ECG stream and performing inference and generating results for each beat in sequence. As shown in Fig. 4.1, the ECG is transmitted from the patch to the smartphone via BLE, while signal reconstruction, R-peak alignment, input construction, classification, and result display are all handled on the device. Fig. 4.2 shows the BLE notification frame structure and ECG sample encoding, and Fig. 4.3 illustrates the smartphone processing flow from received samples to beat-level classification.

4.1 ECG Data Acquisition

The HiCardi® Smart Patch (MEZOO Co., Ltd., Wonju, Republic of Korea) transmits ECG through the Nordic UART Service (NUS) [47]. At the start of acquisition,



(a) (b)

Figure 4.1: HiCardi® Smart Patch and system configuration. (a) Patch and electrode geometry. (b) ECG is streamed over BLE to the smartphone; signal processing, beat classification, and display are performed on device.

the smartphone sequentially sends start commands using the write-without-response operation, without waiting for responses. The patch then periodically transmits 42-byte notifications. Fig. 4.2 shows the notification frame structure and ECG sample encoding. The ECG payload is organized into 3-byte groups in the form `[tag, hi, lo]`, and each 42-byte notification contains 10 ECG samples. Only groups with a valid ECG tag are used to reconstruct the samples. The two 7-bit fields, `hi` and `lo`, are then combined to obtain the 14-bit ADC value:

$$v = (\texttt{hi} \,\&\, \texttt{0x7F})\, 2^7 + (\texttt{lo} \,\&\, \texttt{0x7F}). \tag{4.1}$$

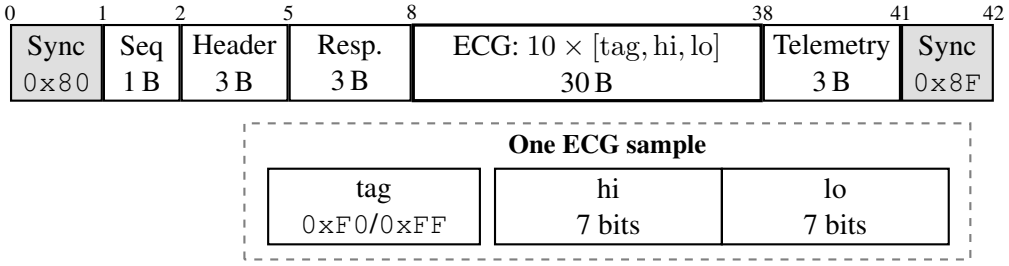


Figure 4.2: BLE frame structure and ECG sample encoding. Each 42-byte notification contains 10 ECG samples represented by `[tag, hi, lo]`, where `hi` and `lo` are 7-bit fields.

The reconstructed ADC value is converted to mV using $x = (v - 8192)/1000$, and the resulting signal, which uses the same scale as the training data, is fed to the model. Sample validity is continuously monitored based on the signal-quality conditions. Lead-off status is determined from the configured damaged-sample ratio and consecutive-frame criteria. During lead-off periods, the ECG waveform continues to be displayed and recorded, while R-peak tracking and inference states are updated accordingly. The signal time axis is generated from sample indices rather than BLE notification arrival times, using a sampling frequency of $f_s = 250\,\mathrm{Hz}$. RR intervals are then calculated from the difference in sample indices between consecutive R-peaks, and packet sequence information is used to track the reception order and identify periods of packet loss.

4.2 On-Device Processing

As shown in Fig. 4.3, smartphone-side processing is divided among a BLE callback, a receive executor, an inference executor, and a UI thread. The BLE callback validates incoming frames and reconstructs the ECG samples. The receive executor handles R-peak alignment and other beat-related processing, while the inference executor constructs beat

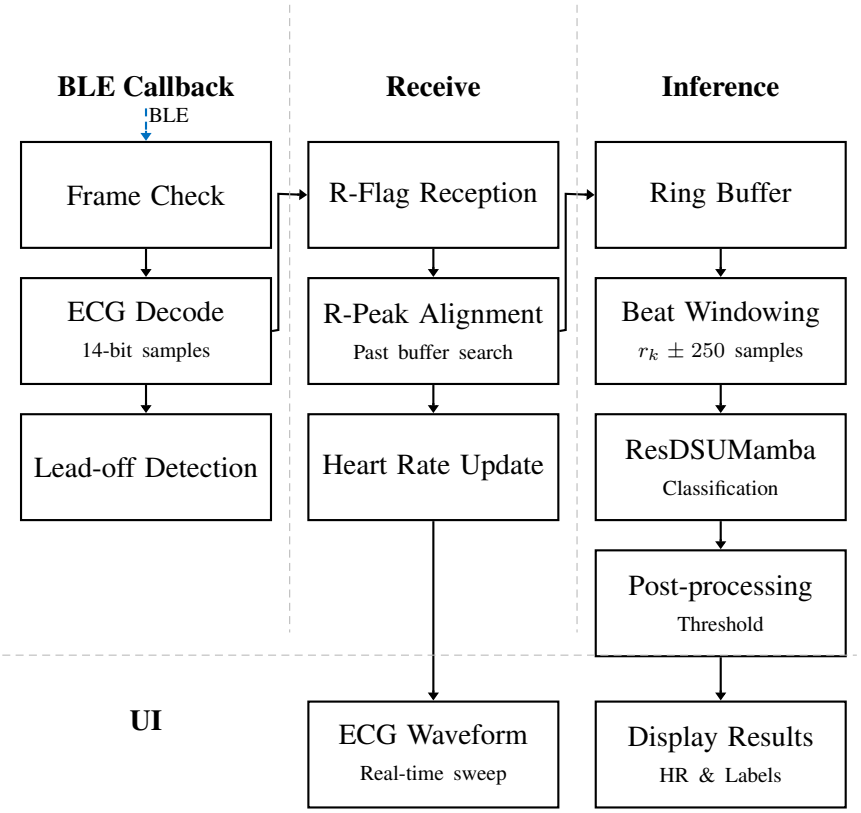


Figure 4.3: End-to-end on-device beat processing. The pipeline is divided into four smartphone-side execution contexts: the BLE callback, receive executor, inference executor, and UI thread. Each block denotes a processing operation, and the queue between the receive and inference executors provides thread-safe beat transfer.

inputs, computes clinical features, and runs ResDSUMamba. Meanwhile, the UI thread continuously updates the ECG waveform and classification results. The receive and inference executors each run on a single thread, with a thread-safe queue used to pass beats from the receive executor to the inference executor.

Reconstructed ECG samples are stored in a fixed-size circular buffer together with their sample indices. For each confirmed R-peak at sample index n_k, a 501-sample beat window $[n_k - W_{\text{pre}},\, n_k + W_{\text{post}}]$ is constructed with $W_{\text{pre}} = W_{\text{post}} = 250$ samples,

that is, $1.0\,\mathrm{s}$ on each side at $f_s = 250\,\mathrm{Hz}$.

The patch provides an R-peak tag for each detected beat, but the tag is temporally offset from the corresponding R-peak in the ECG waveform. Therefore, the tagged sample index is not used directly as the R-peak position. Instead, the receive executor searches the preceding ECG segment and refines the R-peak location from the waveform, with a minimum interval of $240\,\mathrm{ms}$ between consecutive R-peaks. The observed tag-to-waveform offset was approximately $\Delta_{\mathrm{tag}} = 340\,\mathrm{ms}$, and this correction was applied consistently throughout the implementation. This offset represents tag-to-waveform alignment rather than BLE notification transport latency.

The prepared beat is then passed to ResDSUMamba together with the patient embedding $\mathbf{p} \in \mathbb{R}^{256}$ and the clinical feature vector $\mathbf{c} \in \mathbb{R}^{32}$. The patient embedding is obtained from the subject's initial $60\,\mathrm{s}$ baseline ECG using the CPC encoder. Because the beat window extends W_{post} samples beyond the current R-peak, and the clinical features include the post-RR interval, inference can begin only after both the required future ECG samples and the subsequent R-peak are available.

Measured from the R-peak n_k, the time until the input of beat k becomes available is therefore set by whichever condition is met later:

$$D_k = \max\left(RR_k^{\mathrm{post}} + \Delta_{\mathrm{tag}},\ \frac{W_{\mathrm{post}}}{f_s}\right), \tag{4.2}$$

where RR_k^{post} is the interval between the current and subsequent R-peaks. The first term is the time until the tag of the next R-peak arrives, and the second term is the time until the W_{post} post-peak samples are buffered, which is $250/250\,\mathrm{Hz} = 1.0\,\mathrm{s}$. At the mean heart rate of the live session in Section 5.2.3 ($80.2\,\mathrm{bpm}$, so $RR \approx 748\,\mathrm{ms}$), the first term is about $748 + 340 = 1088\,\mathrm{ms}$, so $D_k \approx 1.09\,\mathrm{s}$. This delay is determined by the input

definition and the tag offset, not by computation, and the beat-level processing time is added to it before a result is displayed. Chapter 5 evaluates whether beats are processed without backlog once their inputs are available; D_k itself is derived here analytically and is not measured separately.

Chapter 5

Experiments

5.1 Experimental Setup

5.1.1 Real-World Wearable ECG Dataset

Table 5.1: Cohort composition of the HiCardi records.

		Records				
Cohort	Site	Train	Val.	Test	Records	Beats
01	Local Clinic	42	11	45	98	5,744,567
02	Kosin	11	6	23	40	3,137,371
03	Yonsei Resp. Env.	12	3	26	41	3,867,625
04	Yonsei Resp. Env. 2022a	20	3	23	46	3,638,596
05	Yonsei Resp. Env. 2022c	17	4	21	42	2,222,122
06	Kosin ESKD	17	12	21	50	913,321
Total		119	39	159	317	19,523,602

The HiCardi dataset consists of long-term ECG recordings collected using the HiCardi Smart Patch and includes signal characteristics commonly encountered in real-world wearable monitoring, such as motion artifacts, signal loss, and changes in electrode contact. The single-channel ECG was sampled at $250\,\mathrm{Hz}$, with recording durations

ranging from 0.03 to 47.5 hours. The dataset contains 317 records from 6 cohorts used for training, validation, and evaluation after applying the calibration and signal-quality criteria. Table 5.1 summarizes the composition of each cohort, which are grouped according to acquisition site and recording period.

The five evaluation labels are Normal, APC, VPC, sinus tachycardia, and AF/AFL. APC and VPC represent beat-level morphological labels, whereas sinus tachycardia and AF/AFL represent interval-level rhythm labels. Because a beat can have more than one label, the task was formulated as a multi-label classification problem. Normal denotes a normal heartbeat and was included as a reference label. The initial $60\,\mathrm{s}$ of each subject was used only as an unlabeled baseline segment for patient-embedding extraction and was excluded from performance evaluation.

Table 5.1 reports record counts after the cohort-level inclusion criteria, giving 159 test records. Of these, 21 records did not contain a calibration segment that met the length and signal-quality requirements used for patient-embedding extraction (the initial unlabeled $60\,\mathrm{s}$ segment described above) and were excluded from the record-level evaluation that follows, leaving 138 evaluation records.

The training, validation, and evaluation data were divided at the record level. Among the 78 subjects contributing beats to the evaluation set, 27 also had recordings from different days included in the training data. These overlapping subjects accounted for 87 of the 138 evaluation records and $6{,}146{,}182$ of the $8{,}777{,}480$ evaluation beats ($70.0\,\%$).

5.1.2 Target Device and Measurement

Device measurements were performed on a Samsung SM-G996N (Exynos 2100, arm64-v8a, 8 CPU cores, Android 15, SDK 35). The deployed model used the FP32

XNNPACK configuration of ExecuTorch, batch size 1, and one worker thread. Inference was executed on a Cortex-X1 core using a single thread. The operating frequency was not fixed, and other threads in the actual system handled BLE reception, buffer management, and screen updates. In the deployed graph, operations supported by XNNPACK were delegated to the XNNPACK backend, while unsupported operations were executed by the ExecuTorch portable kernel. The Portable and XNNPACK configurations were compared. T_{model} was measured after 100 warm-up iterations by executing 2,000 forward passes and recording the CPU time used by the inference thread during each forward pass. The mean (Avg), $P5$, $P50$, and $P95$ were all calculated from these 2,000 samples. Peak Native Heap denotes the maximum native allocation observed for the application process during the measurement session.

5.1.3 Evaluation Metrics

The measurements in Section 5.1.2 were obtained by repeatedly executing the model with the input already prepared. During actual streaming, ECG reception, R-peak correction, and inference operate concurrently; therefore, processing time is measured separately for each beat within the application. This work distinguishes the following three timing metrics.

- T_{model}: CPU execution time of one model forward pass given a prepared input, as measured in Section 5.1.2.
- T_{queue}: wall-clock waiting time from the moment the beat input becomes ready until the inference executor begins processing it.
- T_{beat}: total wall-clock elapsed time from input readiness until the corresponding

beat-processing result is generated. This includes input construction, feature computation, model execution, and post-processing.

For beat k, the input-ready time t_k^{ready}, the actual processing start time of the inference executor t_k^{start}, and the result-generation time t_k^{end} are recorded using a monotonic clock. The metrics are calculated as

$$T_{\mathrm{queue},k} = t_k^{\mathrm{start}} - t_k^{\mathrm{ready}}, \qquad T_{\mathrm{beat},k} = t_k^{\mathrm{end}} - t_k^{\mathrm{ready}}. \tag{5.1}$$

T_{queue} and T_{beat} are wall-clock times, whereas T_{model} is CPU time. T_{beat} includes input construction, feature computation, model execution, and post-processing.

Real-time streaming in this work is evaluated as the capability to continuously process sequential beat-level inputs rather than as a measure of end-to-end clinical decision latency. The next beat input typically becomes ready after an interval determined by the subsequent R-peak and the required post-peak samples. Accordingly, sustained operation without backlog requires the beat-processing workload to remain within the available inter-beat processing interval. When beats accumulate, the waiting time T_{queue} increases. Therefore, whether T_{queue} continues to increase throughout the session is also monitored. In this work, T_{queue} is reported only as its maximum value over a session (Section 5.2.3); no within-session time series is plotted, so a gradual upward trend could not be distinguished from occasional spikes using the reported statistic alone.

CPU utilization of the process and major threads and memory usage are recorded at $1\,\mathrm{Hz}$. Each measurement session is logged together with the model file, runtime, backend, numerical precision, batch size, and thread count. The measurement logs are written to a file by a separate logging thread that dequeues records without blocking ECG reception or inference. During measurement, automatic screen-off was disabled to maintain

consistent execution conditions.

Because the HiCardi dataset is multi-label, each label is treated as an independent binary classification problem. For each label, Accuracy (Acc), Sensitivity (Se), Specificity (Sp), positive predictivity (P+), and F_1 are calculated. A label is predicted as positive when its output probability is at least 0.5. Macro metrics are obtained by averaging the metric values of the five labels (Normal, APC, VPC, sinus tachycardia, and AF/AFL) with equal weights. Micro metrics are computed after aggregating the TP, FP, FN, and TN counts across all five labels.

5.2 Experimental Results

5.2.1 Arrhythmia Classification Performance

Table 5.2: Classification performance (%) on the HiCardi test set.

	Macro					Micro				
Model	Acc.	Se	Sp	P+	F_1	Acc.	Se	Sp	P+	F_1
ResUMamba	91.71	78.00	93.66	39.21	42.28	91.71	79.21	94.83	79.32	79.27
ResDSUMamba	93.32	75.72	94.32	41.67	44.15	93.32	83.21	95.85	83.37	83.29

Table 5.2 compares the classification performance of ResUMamba and ResDSUMamba on the HiCardi test set using a record-wise split. The split was configured so that unique record identifiers did not overlap between the training and test sets. The Macro-F_1 and Micro-F_1 values of ResUMamba were 42.28% and 79.27%, respectively, while those of ResDSUMamba were 44.15% and 83.29%, respectively. The differences between the two models were 1.87 percentage points in Macro-F_1 and 4.02 percentage points in Micro-F_1. Macro-Se was 78.00% for ResUMamba and 75.72% for ResD-

SUMamba.

5.2.2 Effect of Software Optimization on Mobile CPU Time

Table 5.3: T_{model} (ms) on Samsung SM-G996N (Exynos 2100) under Portable and XNNPACK configurations.

	ResUMamba					ResDSUMamba				
Backend	Avg	$P5$	$P50$	$P95$	ΔAvg	Avg	$P5$	$P50$	$P95$	ΔAvg
Portable	1073.57	869.08	1211.25	1240.56	0	611.50	537.09	549.20	767.10	0
XNNPACK	48.25	46.56	47.77	51.71	1025.33	**36.70**	**35.59**	**36.11**	**38.90**	574.80

Table 5.3 compares the CPU execution time of one prepared beat input, T_{model}, under two software execution configurations: the ExecuTorch portable-kernel configuration and the XNNPACK configuration. In the portable configuration, model operators are executed using ExecuTorch portable kernels, whereas operations supported by XNNPACK are delegated to the XNNPACK backend in the XNNPACK configuration.

For ResUMamba, the mean T_{model} decreased from $1073.57\,\mathrm{ms}$ with the portable configuration to $48.25\,\mathrm{ms}$ with XNNPACK, corresponding to a reduction of 95.5%. For ResDSUMamba, the mean decreased from $611.50\,\mathrm{ms}$ to $36.70\,\mathrm{ms}$, corresponding to a reduction of 94.0%. Under XNNPACK, ResDSUMamba was additionally $11.55\,\mathrm{ms}$ faster than ResUMamba, corresponding to a relative reduction of 23.9%. The $P95$ of ResDSUMamba was $38.90\,\mathrm{ms}$.

These results show that the software execution configuration had a substantial effect on mobile CPU inference time. The large reduction from the portable configuration to XNNPACK was observed for both models, while the additional use of DSConv further reduced the execution time under the XNNPACK configuration.

5.2.3 Real-World Live Wearable Patch Evaluation

A single session lasting $309.21\,\mathrm{s}$ (approximately 5 min 9 s) was measured while the patch was attached and connected to the smartphone via BLE. The final deployment configuration was evaluated under actual streaming conditions, including ECG reception, R-peak alignment, beat construction, inference, and screen updates. During the measurement, the application performed ECG reception, R-peak alignment, beat classification, and screen updates. The definitions of the timing and resource metrics and the logging procedure are described in Section 5.1.3.

Fig. 5.1 shows the Monitor screen together with CPU and memory utilization during the measurement period. The CPU utilization of the inference thread was 7.9%, that of the reception thread was 1.0%, and Peak Native Heap was $99.2\,\mathrm{MB}$. A total of 413 beats were recorded during the session. Using the measured session duration of $309.2056\,\mathrm{s}$, the beat rate was $413/309.2056 \times 60 = 80.14\,\mathrm{bpm}$. The mean heart rate reported by the application was $80.2\,\mathrm{bpm}$. T_{beat} was $59.4\,\mathrm{ms}$ on average, with a $P50$ of $56.9\,\mathrm{ms}$ and a $P95$ of $70.3\,\mathrm{ms}$. The maximum T_{queue} was $11.4\,\mathrm{ms}$. All beats were classified as Normal.

Table 5.4: Resource usage during continuous ECG streaming with the HiCardi patch on Samsung SM-G996N (Exynos 2100).

Group	Metric	Value
Session	Duration	309.21 s (approx. 5 min 9 s)
	Detected beats N_{R}	413
	Reported heart rate	80.2 bpm
Processing	T_{beat} (Avg.)	59.4 ms
	T_{beat} ($P5$)	49.7 ms
	T_{beat} ($P50$)	56.9 ms
	T_{beat} ($P95$)	70.3 ms
	T_{queue} (max)	11.4 ms
Resource	CPU, inference thread	7.9 %
	CPU, receive thread	1.0 %
	Peak Native Heap	99.2 MB

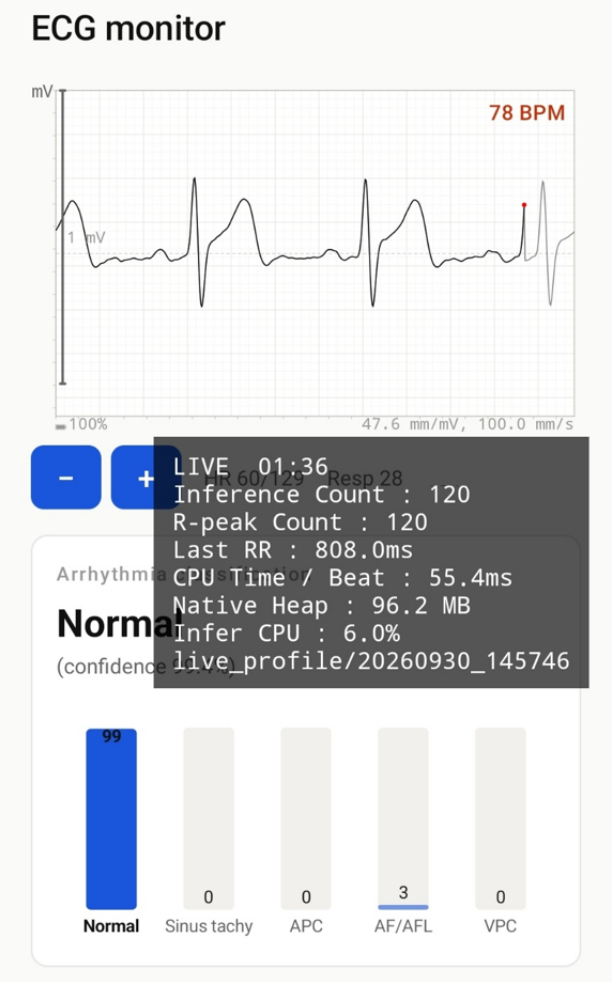


Figure 5.1: Resource usage during live ECG streaming with the HiCardi patch: Monitor screen with the CPU and memory traces recorded on the target device.

Chapter 6

Conclusion

This work developed ResDSUMamba as a lightweight version of ResUMamba and implemented it as an on-device smartphone inference pipeline that classifies beats from a wearable ECG stream in real time. The original ResUMamba combines a ResU path for extracting morphological ECG features with a Mamba path for modeling temporal dependencies. For Android CPU deployment, this work uses a deployment-oriented sequential scan based on standard ATen operators and optimizes both the model architecture and its computational representation.

First, ResDSUMamba was constructed by replacing the standard one-dimensional convolutions in the ResU path with depthwise separable convolutions and retraining the resulting model under the same data split and classification objective. The Mamba path uses the weights of the trained ResDSUMamba, and the selective scan is represented using a sequential scan composed of standard PyTorch operators before deployment through ExecuTorch. For real-time execution, the ECG processing pipeline was implemented to handle ECG reception, 14-bit sample reconstruction, circular-buffer management, R-peak alignment, beat construction, and model inference. Reception and inference were executed on separate threads. Signal quality, lead-off status, and packet loss

were monitored during reception, and R-peak tracking and inference states were updated accordingly.

ResDSUMamba reduced the number of parameters from $2.4883\,\mathrm{M}$ to $1.3225\,\mathrm{M}$, corresponding to a reduction of 46.9%, and reduced the number of MACs from $721.50\,\mathrm{M}$ to $331.38\,\mathrm{M}$, corresponding to a reduction of 54.1%. On the HiCardi dataset using a record-wise split, the Macro-F_1 and Micro-F_1 values of ResUMamba were 42.28% and 79.27%, respectively, whereas those of ResDSUMamba were 44.15% and 83.29%. The corresponding Macro-Se values were 78.00% and 75.72%, so the gain in F_1 came with a lower average sensitivity across labels.

On the Galaxy S21+, the mean T_{model} under the XNNPACK configuration was $48.25\,\mathrm{ms}$ for ResUMamba and $36.70\,\mathrm{ms}$ for ResDSUMamba. Most of the reduction from the portable configuration came from XNNPACK delegation (95.5% for ResUMamba), and DSConv reduced the mean execution time by a further $11.55\,\mathrm{ms}$, corresponding to a relative reduction of 23.9%. The $P95$ of ResDSUMamba was $38.90\,\mathrm{ms}$.

During actual HiCardi patch streaming, 413 beats were processed over $309.21\,\mathrm{s}$, and the application reported a mean heart rate of $80.2\,\mathrm{bpm}$. T_{beat} was $59.4\,\mathrm{ms}$ on average, with a $P95$ of $70.3\,\mathrm{ms}$, and the maximum T_{queue} was $11.4\,\mathrm{ms}$. The CPU utilization of the inference thread was 7.9%, and Peak Native Heap was $99.2\,\mathrm{MB}$. These results show that the on-device streaming pipeline was able to process wearable patch ECG on the smartphone on a beat-by-beat basis without backlog under the measurement conditions used in this study.

6.1 Limitations and Future Work

The on-device evaluation in this work was conducted in a single $309.21\,\mathrm{s}$ session on a Galaxy S21+. Although T_{model} and T_{beat} were measured, the input-availability delay of about $1.09\,\mathrm{s}$ was derived analytically, and end-to-end latency including BLE communication and screen rendering, power consumption, and thermal behavior during prolonged operation were not evaluated. Future work will extend the evaluation to a wider range of heart-rate conditions, longer-duration measurements, and multiple mobile devices.

The live session did not include synchronized clinical ground-truth labels, and all beats in the session were classified as Normal. In this setting, processing time and resource usage were measured, but classification performance could not be directly evaluated against clinical annotations. Future work will compare the deployed model with physician annotations from actual arrhythmia patients.

The HiCardi dataset is not publicly available to protect patient privacy. However, access may be granted for academic and research purposes through the predefined approval procedures established by the data management authority.

Numerical agreement between the PyTorch training-time model and its ExecuTorch-exported graph was not verified. The classification performance in Table 5.2 is measured on the PyTorch model, and the on-device timing in Table 5.3 and Table 5.4 is measured on the exported graph; the two are not linked by a reported output-parity check, so the exported graph's classification performance is assumed, not demonstrated, to match Table 5.2.

References

[1] Philip A Wolf, Robert D Abbott, and William B Kannel, "Atrial fibrillation as an independent risk factor for stroke: the framingham study," *Stroke*, vol. 22, no. 8, pp. 983–988, 1991.

[2] Steven R Steinhubl, Jill Waalen, Alison M Edwards, Lauren M Ariniello, Rajesh R Mehta, Gail S Ebner, Chureen Carter, Katie Baca-Motes, Elise Felicione, Troy Sarich, et al., "Effect of a home-based wearable continuous ecg monitoring patch on detection of undiagnosed atrial fibrillation: The mstops randomized clinical trial," *JAMA*, vol. 320, no. 2, pp. 146–155, 2018.

[3] Marco V Perez, Kenneth W Mahaffey, Haley Hedlin, John S Rumsfeld, Ariadna Garcia, Todd Ferris, Vidhya Balasubramanian, et al., "Large-scale assessment of a smartwatch to identify atrial fibrillation," *New England Journal of Medicine*, vol. 381, no. 20, pp. 1909–1917, 2019.

[4] Philip de Chazal, Maria O'Dwyer, and Richard B Reilly, "Automatic classification of heartbeats using ecg morphology and heartbeat interval features," *IEEE transactions on biomedical engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.

[5] Farid Melgani and Yakoub Bazi, "Classification of electrocardiogram signals with support vector machines and particle swarm optimization," *IEEE transactions on information technology in biomedicine*, vol. 12, no. 5, pp. 667–677, 2008.

[6] Eduardo José da S Luz, William Robson Schwartz, Guillermo Cámara-Chávez, and David Menotti, "Ecg-based heartbeat classification for arrhythmia detection: A survey," *Computer methods and programs in biomedicine*, vol. 127, pp. 144–164, 2016.

[7] Sandeep Raj and Kailash Chandra Ray, "Ecg signal analysis using dct-based dost and pso optimized svm," *IEEE Transactions on instrumentation and measurement*, vol. 66, no. 3, pp. 470–478, 2017.

[8] Jinghao Niu, Yongqiang Tang, Zhengya Sun, and Wensheng Zhang, "Inter-patient ecg classification with symbolic representations and multi-perspective convolutional neural networks," *IEEE journal of biomedical and health informatics*, vol. 24, no. 5, pp. 1321–1332, 2019.

[9] Enbiao Jing, Haiyang Zhang, ZhiGang Li, Yazhi Liu, Zhanlin Ji, and Ivan Ganchev, "Ecg heartbeat classification based on an improved resnet-18 model," *Computational and Mathematical Methods in Medicine*, vol. 2021, no. 1, pp. 6649970, 2021.

[10] Saeed Saadatnejad, Mohammadhosein Oveisi, and Matin Hashemi, "Lstm-based ecg classification for continuous monitoring on personal wearable devices," *IEEE journal of biomedical and health informatics*, vol. 24, no. 2, pp. 515–523, 2019.

[11] Borui Hou, Jianyong Yang, Pu Wang, and Ruqiang Yan, "Lstm-based auto-encoder model for ecg arrhythmias classification," *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 4, pp. 1232–1240, 2019.

[12] Yongbin Lee and Ki H Chon, "Atrial fibrillation prediction using a lightweight temporal convolutional and selective state space architecture," in *2025 IEEE 21st International Conference on Body Sensor Networks (BSN)*. IEEE, 2025, pp. 1–4.

[13] Sneha Bathini, "Mamba differential selective state space modelling framework for robust continuous ecgbased arrhythmia classification," in *2026 5th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*. IEEE, 2026, pp. 1–8.

[14] Md Rabiul Islam, Marwa Qaraqe, Khalid Qaraqe, and Erchin Serpedin, "Cat-net: Convolution, attention, and transformer based network for single-lead ecg arrhythmia classification," *Biomedical Signal Processing and Control*, vol. 93, pp. 106211, 2024.

[15] Chao Che, Peiliang Zhang, Min Zhu, Yue Qu, and Bo Jin, "Constrained transformer network for ecg signal processing and arrhythmia classification," *BMC medical informatics and decision making*, vol. 21, no. 1, pp. 184, 2021.

[16] Mariano Llamedo and Juan Pablo Martínez, "Heartbeat classification using feature selection driven by database generalization criteria," *IEEE Transactions on Biomedical Engineering*, vol. 58, no. 3, pp. 616–625, 2011.

[17] Yeji Jeong, Jaewon Lee, and Miyoung Shin, "Enhancing inter-patient performance for arrhythmia classification with adversarial learning using beat-score maps," *Applied Sciences*, vol. 14, no. 16, pp. 7227, 2024.

[18] Sean Shensheng Xu, Man-Wai Mak, and Chi-Chung Cheung, "I-vector-based patient adaptation of deep neural networks for automatic heartbeat classification," *IEEE Journal of Biomedical and Health Informatics*, vol. 24, no. 3, pp. 717–727, 2020.

[19] Junyeong Heo, Jaebin Cha, Youngcheol Park, Sung Pil Cho, and Minwoo Kim, "Patient-conditioned ecg beat classification via self-supervised embeddings for robust inter-patient arrhythmia detection," *Expert Systems with Applications*, vol. 331, pp. 133149, 2026.

[20] Wei Fan, Yujuan Si, Meiqi Sun, Lin Zhou, Weiyi Yang, Adi Alhudhaif, and Fayadh Alenezi, "Class-specific weighted broad learning system-based domain adaptation for patient-specific ecg classification," *Expert Systems with Applications*, vol. 273, pp. 126824, 2025.

[21] Albert Gu and Tri Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[22] Yeonjae Park, You Hyun Park, Hoyeon Jeong, Kise Kim, Ji Ye Jung, Jin-Bae Kim, and Dae Ryong Kang, "Development and validation of a real-time service model for noise removal and arrhythmia classification using electrocardiogram signals," *Sensors*, vol. 24, no. 16, pp. 5222, 2024.

[23] Mariano Llamedo and Juan Pablo Martínez, "An automatic patient-adapted ecg heartbeat classifier allowing expert assistance," *IEEE Transactions on Biomedical Engineering*, vol. 59, no. 8, pp. 2312–2320, 2012.

[24] Philip De Chazal and Richard B Reilly, "A patient-adapting heartbeat classifier using ecg morphology and heartbeat interval features," *IEEE transactions on biomedical engineering*, vol. 53, no. 12, pp. 2535–2543, 2006.

[25] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, 2015, pp. 234–241.

[26] Zhengxin Zhang, Qingjie Liu, and Yunhong Wang, "Road extraction by deep residual U-Net," *IEEE Geoscience and Remote Sensing Letters*, vol. 15, no. 5, pp. 749–753, 2018.

[27] Albert Gu, Karan Goel, and Christopher Re, "Efficiently modeling long sequences with structured state spaces," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[28] Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré, "On the parameterization and initialization of diagonal state space models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 35971–35983, 2022.

[29] Aaron van den Oord, Yazhe Li, and Oriol Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[30] Serkan Kiranyaz, Turker Ince, and Moncef Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016.

[31] Antonino Faraone and Ricard Delgado-Gonzalo, "Convolutional-recurrent neural networks on low-power wearable platforms for cardiac arrhythmia detection," in *2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2020, pp. 153–157.

[32] Thorir Mar Ingolfsson, Xiaying Wang, Michael Hersche, Alessio Burrello, Lukas Cavigelli, and Luca Benini, "ECG-TCN: Wearable cardiac arrhythmia detection with a temporal convolutional network," in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2021, pp. 1–4.

[33] Kwang-Sig Lee, Hyun-Joon Park, Ji Eon Kim, Hee Jung Kim, Sangil Chon, Sangkyu Kim, Jaesung Jang, Jin-Kook Kim, Seongbin Jang, Yeongjoon Gil, and Ho Sung Son, "Compressed deep learning to classify arrhythmia in an embedded wearable device," *Sensors*, vol. 22, no. 5, pp. 1776, 2022.

[34] Xinzi Xu, Yanxing Suo, Qiao Cai, Yunfang Zhang, Qinyu Chen, Chang Gao, Guoxing Wang, and Yang Zhao, "Cardiolike-net: An edge-end inter-patient arrhythmia classifier with quantization-aware-training for wearable ecg applications," *Biomedical Signal Processing and Control*, vol. 112, pp. 108604, 2026.

[35] Google LLC, "TensorFlow Lite (LiteRT): On-device machine learning framework," `https://ai.google.dev/edge/litert`, 2025, Accessed: 2026-09.

[36] PyTorch Foundation, "ExecuTorch documentation," https://docs.pytorch.org/executorch/, 2025, Accessed: 2026-09.

[37] Microsoft, "ONNX Runtime: Cross-platform, high performance ml inferencing and training accelerator," https://onnxruntime.ai, 2025, Accessed: 2026-09.

[38] Google LLC, "XNNPACK: High-efficiency floating-point neural network inference operators for mobile, server, and web," https://github.com/google/XNNPACK, 2025, Accessed: 2026-09.

[39] W. Daniel Hillis and Guy L. Steele, Jr., "Data parallel algorithms," *Communications of the ACM*, vol. 29, no. 12, pp. 1170–1183, 1986.

[40] Guy E. Blelloch, "Prefix sums and their applications," Tech. Rep. CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, 1990.

[41] Mark Harris, Shubhabrata Sengupta, and John D. Owens, "Parallel prefix sum (scan) with CUDA," in *GPU Gems 3*, Hubert Nguyen, Ed., chapter 39, pp. 851–876. Addison-Wesley, 2007.

[42] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[43] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[44] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[45] Hongjun Xu, Junxi Xia, Weisi Yang, Yueyuan Sui, and Stephen Xia, "MambaLite-Micro: Memory-optimized Mamba inference on MCUs," *arXiv preprint arXiv:2509.05488*, 2025.

[46] Arghadip Das, Arnab Raha, Shamik Kundu, Soumendu Kumar Ghosh, Deepak Mathaikutty, and Vijay Raghunathan, "XAMBA: Enabling efficient state space models on resource-constrained neural processing units," in *ICLR Workshop on Scalable Optimization for Efficient and Adaptive Foundation Models (SCOPE)*, 2025, arXiv:2502.06924.

[47] Nordic Semiconductor, "Nordic UART Service (NUS), nRF Connect SDK documentation," `https://docs.nordicsemi.com/`, 2025, Accessed: 2026-09.

Appendix A

Evaluation of Pruning for Model Lightweighting

This appendix reports the pruning experiments conducted on the MIT-BIH Arrhythmia Database and clarifies the rationale for adopting Depthwise Separable Convolution (DSConv) for the final mobile deployment. Classes follow AAMI EC57: N (normal and bundle-branch block), S (supraventricular ectopic), V (ventricular ectopic), and F (fusion). These experiments use models trained on MIT-BIH with the AAMI EC57 label set, which differs from the HiCardi multi-label setting used elsewhere in this thesis; the latencies reported below are therefore not directly comparable to the HiCardi-trained, final-deployment-graph latencies in Table 5.3.

Pruning was initially evaluated as an alternative method to reduce the model size. Fig. A.1 shows the effect of unstructured (left) and structured (right) pruning on classification performance. After fine-tuning (bottom row), unstructured pruning maintains the weighted F_1 score up to a ratio of 0.7. However, structured pruning degrades sharply from the first non-zero ratio, settling near a trivial majority-class baseline (labeling every beat N yields an F_1 of $82.1\,\%$).

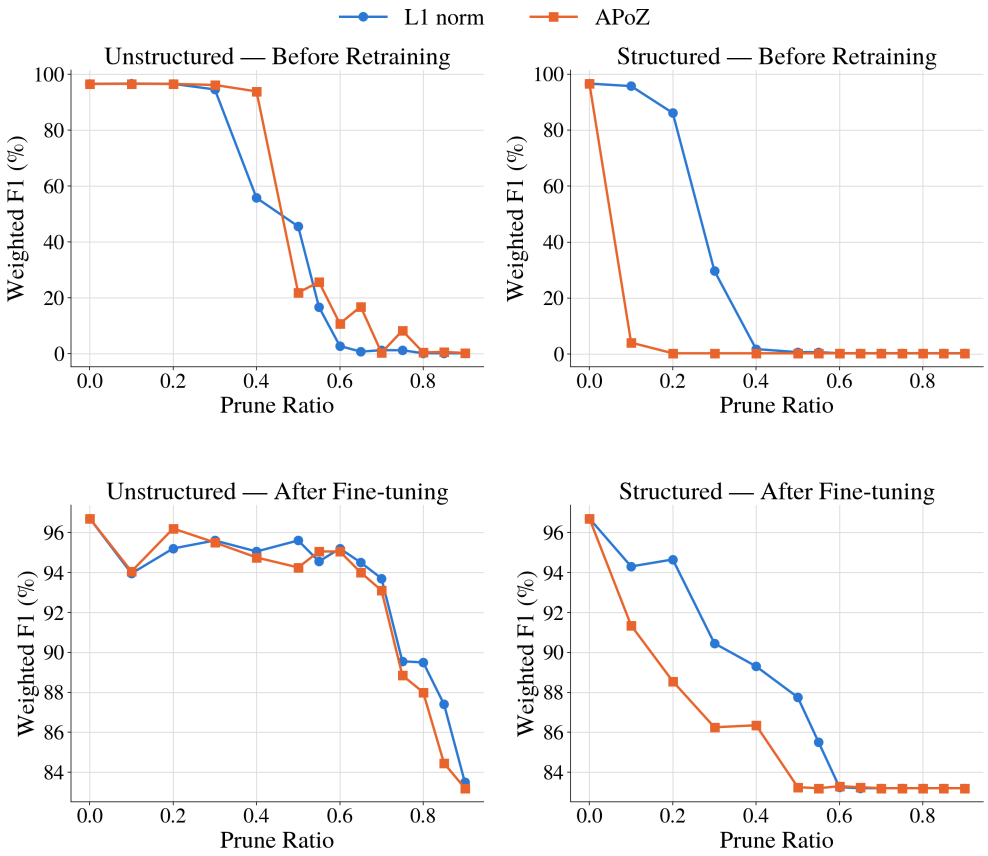
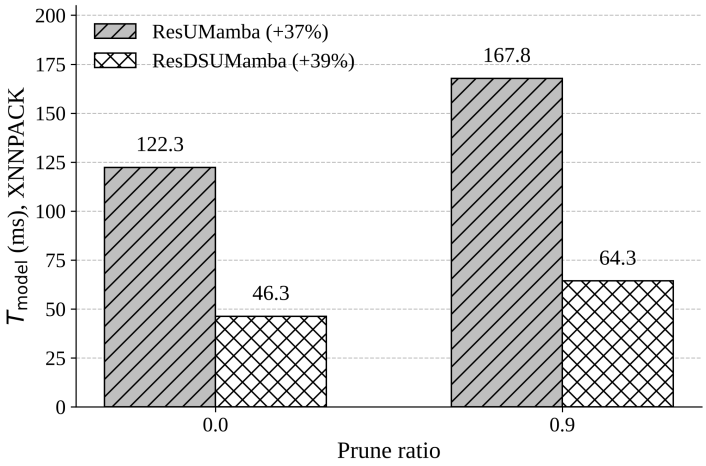


Figure A.1: Weighted F_1 versus prune ratio on MIT-BIH. Left: unstructured pruning; right: structured pruning. Top: before retraining; bottom: after fine-tuning.

Sparse representations can theoretically reduce storage, but this does not imply a corresponding reduction in mobile CPU latency. As shown in Fig. A.2, increasing the prune ratio did not result in faster execution. On the target handset with the XNNPACK delegate, the mean latency actually increased between ratios of 0.0 and 0.9: by $37\,\%$ for ResUMamba (122.3 to $167.8\,\mathrm{ms}$) and by $39\,\%$ for ResDSUMamba (46.3 to $64.3\,\mathrm{ms}$). The exported model files maintained the same size at both ratios, indicating that the pruned weights were stored as zeros in dense tensors, thus failing to reduce the actual



Only ratios 0.0 and 0.9 were measured on-device (MIT-BIH models, XNNPACK, target handset). Intermediate ratios (0.1-0.8) were not measured; no interpolation is shown. Percentages are the 0.0->0.9 latency increase quoted in the Appendix text.

Figure A.2: Mobile CPU inference latency versus prune ratio on the target mobile handset using the XNNPACK delegate. Latency increases despite the increased prune ratio.

operations executed on the device.

Because structured pruning loses accuracy immediately and unstructured pruning fails to provide a useful reduction in computation for the target mobile deployment, pruning was deemed unsuitable. Consequently, the depthwise separable convolution (DSConv) discussed in Chapter 3 was adopted to provide a structurally reduced convolutional architecture with lower MACs and improved suitability for mobile CPU execution.

웨어러블 심전도 모니터링을 위한 경량 ResUMamba 기반 온디바이스 실시간 부정맥 분류

Ryuha Kim
전산학과
연세대학교 일반대학원

웨어러블 ECG patch는 일상생활에서 간헐적으로 발생하는 부정맥을 장기간 모니터링할 수 있지만, 장시간 기록을 모두 수동으로 판독하기에는 한계가 있으며, 클라우드 기반 분석은 추가적인 통신 및 데이터 처리 경로를 필요로 한다. 본 논문에서는 환자 조건화 경로를 포함하는 기존 ResUMamba를 스마트폰에서 실행하기 위해 모델을 경량화하고 CPU 중심의 배포 방법을 적용하였다. 새로운 분류 구조를 제안하는 대신, ResU path의 표준 합성곱을 depthwise separable convolution으로 대체하고 decoder의 transposed convolution을 파라미터 없는 선형 보간으로 대체하여 ResDSUMamba를 구성하였다. 또한 Mamba path의 selective scan은 PyTorch의 표준 ATen 연산을 이용해 다시 작성하고, 고정된 입력 길이 $T = 501$에 대해 순차적으로 펼친 정적 그래프로 구성하여 배포하였다.

ResDSUMamba는 1.3225 M개의 파라미터와 331.38 M MAC을 가지며, 기존 ResUMamba의 2.4883 M 및 721.50 M과 비교하여 각각 46.9%와 54.1% 감소하였다. HiCardi 데이터셋의 레코드 단위 분할에서 ResDSUMamba의 Macro-F_1과 Micro-F_1은 각각 44.15%와 83.29%로, ResUMamba의 42.28%와 79.27%보다 높았으며, Macro-Se는 78.00%에서 75.72%로 낮아졌다. Galaxy S21+(Exynos 2100)의 Cortex-X1 단일 스레드 FP32 조건에서 XNNPACK 위임은 ResUMamba의 평균 모델 실행시간을 1073.57 ms에서 48.25 ms로 줄였고, DSConv를 적용한 ResDSUMamba는 36.70 ms(P95 38.90 ms)였다. 실제 patch 연동 스트리밍에서는 309.21 s 동안 413개의 beat가 대기열 누적 없이 처리되었고, 애플리케이션에서 보고된 평균 심박수는 80.2 bpm이었다. 심박당 처리시간은 평균 59.4 ms, P95 70.3 ms였으며, 추론 스레드의 CPU 사용률은 7.9%, Peak Native Heap은 99.2 MB로 측정되었다. 실기기 세션에는 기준 라벨이 없어 온디바이스 분류 정확도는 평가하지 않았다.

핵심되는 말 : 심전도(ECG), 부정맥 분류, 상태 공간 모델(SSMs), 온디바이스 추론, 모바일 CPU 최적화, 웨어러블 헬스케어